Email Spam Classifier — Project Report

Project Title: Email Spam Classifier using Machine Learning
Algorithm: Multinomial Naive Bayes with TF-IDF Vectorization
Interface: Flask Web Application with Interactive Dashboard
Language: Python 3

Table of Contents

1. Project Overview
2. Objectives
3. Technologies Used
4. Dataset
5. System Architecture
6. Machine Learning Pipeline
7. Model Details
8. Performance Results
9. Web Application
10. Key Features
11. Project Structure
12. How to Run
13. Conclusion
14. Future Improvements

---

1. Project Overview

This project is an end-to-end **Email Spam Classifier** that uses **Machine Learning** to automatically detect whether a given email or SMS message is spam or legitimate (ham). The classifier is powered by a **Multinomial Naive Bayes** model trained on TF-IDF features extracted from real message data.

The project includes:

- A complete offline **training pipeline** (`train.py`)
- A **command-line prediction tool** (`predict.py`)
- A **Flask web application** (`app.py`) serving a modern interactive dashboard
- **Real-time spam alerting** with visual and audio warnings

---

2. Objectives

- Build a machine learning model that accurately classifies messages as spam or ham
- Apply standard NLP preprocessing techniques including stopword removal and stemming
- Use TF-IDF vectorization to convert text into numerical features
- Train and evaluate a Multinomial Naive Bayes classifier
- Deploy the model as a live web application with a user-friendly dashboard

- Alert users immediately when spam is detected through visual and audio notifications

---

3. Technologies Used

Category	Technology	Purpose
Language	Python 3	Core development
ML Framework	scikit-learn	Model training and evaluation
NLP	NLTK	Stopword removal, stemming
Vectorization	TF-IDF (scikit-learn)	Text to feature matrix
Data Handling	Pandas, NumPy	Dataset loading and manipulation
Visualization	Matplotlib, Seaborn	Confusion matrix heatmap
Web Framework	Flask	REST API and web server
Frontend	HTML5, CSS3, Vanilla JS	Interactive dashboard UI
Charts	Chart.js	Performance bar chart
PDF Generation	ReportLab	Downloadable model report
Model Persistence	Joblib	Save/load model and vectorizer
Fonts	Google Fonts (Inter, JetBrains Mono)	Dashboard typography

---

4. Dataset

Name: SMS Spam Collection Dataset
Source: UCI Machine Learning Repository
Format: CSV with two columns — label (`ham`/`spam`) and message text

Statistic	Value
Total Messages	5,574
Ham (Legitimate)	4,827 (86.6%)
Spam	747 (13.4%)
Training Set (80%)	4,459 samples
Test Set (20%)	1,115 samples

The dataset is a well-known benchmark for spam classification research. It contains real SMS messages tagged as ham or spam, covering a wide variety of spam patterns including prize scams, urgent offers, phishing links, and fake bank alerts.

---

5. System Architecture

```
┌──────────────────────────────────────────────────────┐
│                  TRAINING PHASE                      │
│                                                      │
│  spam.csv  →  Preprocessing  →  TF-IDF  →  80/20     │
│                                          Split       │
│                    ↓                                 │
│          MultinomialNB.fit()                         │
│                    ↓                                 │
│    model.pkl  +  vectorizer.pkl  +  metrics.json     │
│                  confusion_matrix.png                │
└──────────────────────────────────────────────────────┘

┌──────────────────────────────────────────────────────┐
│                PREDICTION PHASE                      │
│                                                      │
│  User Input  →  Preprocessing  →  vectorizer         │
│               .transform()     →  model.predict()    │
│                                          ↓           │
│               Result + Confidence Score              │
│                        ↓                             │
│         If SPAM → Visual Alert + Audio Alarm         │
└──────────────────────────────────────────────────────┘

┌──────────────────────────────────────────────────────┐
│                 FLASK WEB APP                        │
│                                                      │
│   GET  /              → Dashboard (index.html)       │
│   POST /predict       → JSON prediction response     │
│   GET  /metrics       → Model metrics JSON           │
│   GET  /download-report → PDF report (ReportLab)     │
└──────────────────────────────────────────────────────┘
```

---

6. Machine Learning Pipeline

Step 1 — Data Loading

The dataset is loaded from `dataset/spam.csv` using Pandas with Latin-1 encoding to handle special characters. Unnamed columns are dropped automatically.

Step 2 — Text Preprocessing

Each message goes through a four-stage cleaning pipeline:

```
Raw Text
   ↓  Convert to lowercase
   ↓  Remove non-alphabetic characters (regex: [^a-zA-Z\s])
   ↓  Tokenize by splitting on whitespace
   ↓  Remove English stopwords (NLTK stopwords corpus)
```

```
    ↓  Apply Porter Stemming (reduces words to root form)
Clean Text
```

Example:

- Input: `"Congratulations!! You WON £50,000 — Click HERE to claim!!"`
- Output: `"congratul won click claim"`

Step 3 — TF-IDF Vectorization

`TfidfVectorizer(max_features=5000)` converts the cleaned text corpus into a numerical feature matrix.

- **TF (Term Frequency):** How often a word appears in a message
- **IDF (Inverse Document Frequency):** Penalises words that appear in almost all messages
- **Result:** 5,574 × 5,000 sparse matrix

Step 4 — Train-Test Split

The dataset is split 80% / 20% with `random_state=42` for reproducibility:

- Training: 4,459 samples
- Testing: 1,115 samples

Step 5 — Model Training

`MultinomialNB()` is trained on the TF-IDF feature matrix. Multinomial Naive Bayes is the industry-standard choice for text classification because:

- It is computationally efficient on sparse matrices
- It naturally handles word count distributions
- It produces calibrated probability scores (confidence)
- It performs very well even on smaller datasets

Step 6 — Evaluation

The model is evaluated on the held-out test set using four standard classification metrics.

Step 7 — Artifact Persistence

All artifacts are saved to disk:

- `model.pkl` — trained classifier
- `vectorizer.pkl` — fitted TF-IDF vectorizer
- `metrics.json` — evaluation scores and dataset stats
- `static/confusion_matrix.png` — heatmap visualization

---

7. Model Details

Parameter	Value
Algorithm	Multinomial Naive Bayes
Vectorizer	TF-IDF
Max Features	5,000
Train/Test Split	80% / 20%
Random State	42
Smoothing (alpha)	1.0 (default Laplace)
Input	Raw email/SMS text
Output	Label (Spam/Ham) + Confidence %

Preprocessing pipeline (identical at train and inference time):

1. Lowercase conversion
2. Non-alphabetic character removal
3. NLTK English stopword removal
4. Porter Stemming

---

8. Performance Results

Evaluation Metrics

Metric	Score
Accuracy	**97.22%**
Precision	**99.24%**
Recall	**81.37%**
F1 Score	**89.42%**

Confusion Matrix

```
                 Predicted Ham    Predicted Spam
Actual Ham             953               1
Actual Spam             30             131
```

Cell	Value	Meaning
True Negative (TN)	953	Ham correctly identified as ham
False Positive (FP)	1	Ham incorrectly flagged as spam
False Negative (FN)	30	Spam missed (classified as ham)

Cell	Value	Meaning
True Positive (TP)	131	Spam correctly detected

Metric Interpretation

- **Accuracy (97.22%):** Out of 1,115 test messages, 1,084 were classified correctly.
- **Precision (99.24%):** When the model says spam, it is correct 99.24% of the time. Only 1 legitimate message was incorrectly flagged — extremely low false positive rate.
- **Recall (81.37%):** The model caught 81.37% of all actual spam messages. 30 spam messages were missed.
- **F1 Score (89.42%):** The harmonic mean of precision and recall — a balanced measure of overall performance.

The high precision (99.24%) is particularly important in this context because falsely marking a legitimate email as spam is more disruptive to users than occasionally missing a spam message.

---

9. Web Application

The Flask application (`app.py`) serves a modern single-page dashboard with the following API endpoints:

Endpoints

Method	Route	Description
GET	`/`	Serves the main dashboard HTML page
POST	`/predict`	Accepts `{"email": "..."}` JSON, returns prediction result, is_spam flag, and confidence score
GET	`/metrics`	Returns model metrics from `metrics.json` as JSON
GET	`/download-report`	Generates and streams a formatted A4 PDF report

Prediction Response

```
{
  "result": "Spam Email",
  "is_spam": true,
  "confidence": 98.1
}
```

---

10. Key Features

Dashboard

- **Email Input** — textarea with character counter and Ctrl+Enter shortcut

- **Sample message chips** — 4 quick-fill buttons (2 spam, 2 ham) for instant demo
- **Result display** — animated box showing Spam/Ham label with confidence percentage
- **Session Log** — scrollable history of all predictions made in the current session, shown as individual card rows directly below the input
- **Confusion Matrix** — interactive HTML table + seaborn heatmap image
- **Performance Chart** — Chart.js bar chart showing all 4 metrics
- **Light / Dark theme** — toggle persisted in localStorage
- **Fully responsive** — works on desktop and mobile

Spam Alert System

When a spam message is detected, the dashboard triggers three simultaneous alerts:

1. **Screen Flash** — The screen flashes red twice instantly
2. **Audio Alarm** — A repeating 2-tone alarm generated via the Web Audio API (no audio file required)
3. **Full-screen Modal Overlay** with:
    - Pulsing 🚨 icon with bounce animation and red glow
    - "SPAM DETECTED" heading
    - Confidence score display
    - Three safety tips: do not reply, avoid links, never share passwords
    - Dismiss button and mute/unmute toggle
    - Keyboard (Escape) and backdrop click to dismiss

CLI Tool (`predict.py`)

A standalone command-line REPL that loads the saved model and lets users classify messages interactively without the web interface.

---

11. Project Structure

```
ml_prroject/
│
├── dataset/
│   └── spam.csv                  # SMS Spam Collection (5,574 rows)
│
├── static/
│   ├── style.css                 # Full dashboard styling (light + dark)
│   └── confusion_matrix.png      # Generated heatmap (auto-created)
│
├── templates/
│   └── index.html                # Single-page dashboard
│
├── train.py                      # ML training pipeline
├── predict.py                    # CLI prediction tool
├── app.py                        # Flask web application
├── requirements.txt              # Python dependencies
│
├── model.pkl                     # Saved trained model (auto-generated)
```

```
├── vectorizer.pkl                # Saved TF-IDF vectorizer (auto-generated)
└── metrics.json                  # Saved evaluation metrics (auto-generated)
```

---

12. How to Run

Prerequisites

- Python 3.8 or higher
- pip

Installation

```
# 1. Install dependencies
pip install -r requirements.txt

# 2. Train the model (generates model.pkl, vectorizer.pkl, metrics.json)
python train.py

# 3. Launch the web dashboard
python app.py
```

Open your browser at: **http://127.0.0.1:5000**

CLI Usage (optional)

```
python predict.py
```

Dependencies (`requirements.txt`)

```
pandas
numpy
scikit-learn
matplotlib
seaborn
flask
nltk
joblib
reportlab
```

---

13. Conclusion

This project successfully demonstrates a complete end-to-end machine learning system for email spam detection. The Multinomial Naive Bayes classifier trained on TF-IDF features achieves **97.22% accuracy** and

99.24% precision on the SMS Spam Collection benchmark dataset — results consistent with published research on this dataset.

The project goes beyond a basic ML notebook by integrating the trained model into a production-style web application with real-time prediction, a session history log, interactive visualizations, and an innovative spam alerting system that notifies users with both visual and audio cues.

Key achievements:

- Near-zero false positive rate (only 1 legitimate message misclassified out of 954)
- 97%+ overall accuracy on a real-world 5,574-message benchmark
- Full-stack implementation from data preprocessing to live web deployment
- User-friendly dashboard with spam alert system for immediate threat awareness

---

14. Future Improvements

Improvement	Description
Deep Learning	Replace Naive Bayes with LSTM or BERT for better context understanding
More Features	Include email headers, sender reputation, and URL analysis
Multi-language	Extend to non-English spam detection
User Feedback	Allow users to report false positives/negatives to improve the model
Model Retraining	Add an admin route to retrain on new data without restarting the server
Database Logging	Store all predictions in a database for analytics and model monitoring
API Authentication	Add API key authentication for the `/predict` endpoint
Dockerization	Package the entire app in a Docker container for easy deployment
SVM Comparison	Benchmark against Support Vector Machine and Logistic Regression

---

Report generated for the Email Spam Classifier project.
Model trained on the UCI SMS Spam Collection Dataset.